General API Architecture, Integration and Porting Guide.

Rev 1.0.1

Table of Contents

Revision History

08 / 04 / 2020 – Rev: 0.0.1	Initial Document.
09 / 14 / 2020 – Rev: 0.0.2	Added device handle and HAL function for GPIO control.
11 / 06 / 2020 – Rev: 0.0.3	Updated description for integration and build section.
12 / 01 / 2020 – Rev: 0.0.4	Added integration and build info. for both SDP-K1 and ADS9/x Platforms.
12 / 21 / 2020 – Rev: 1.0.0	Initial Document Release.
01 / 12 / 2021 – Rev: 1.0.1	Fixed a typographical error for file location

Introduction

Purpose

This document serves as a programmer's reference for using and utilizing various aspects of the Application Program Interface (API) library targeting various ADI products. It describes the general structure of the API library, provides a detail list of the API functions and its associated data structures, macros, and definitions.

Scope

This document targets API libraries for most of the High-Speed RF converters. It provides an overview on the API architecture and some common source files which are shared by most of the ADI device for which a standalone API will be provided. Along with this generic guide user should also refer to the product specific API guide, which covers APIs specifically written to control that chip.

Disclaimer

This is a preliminary pre-release version of API and documentation. All information is subject to change.

The software and any related information and/or advice is provided on and "AS IS" basis, without representations, guarantees or warranties of any kind, express or implied, oral or written, including without limitation warranties of merchantability fitness for a particular purpose, title and non-infringement. Please refer to the Software License Agreement applied to the source code for full details.

Software Architecture

The API library provides consistent interface between the ADI device and various client platforms. The library is a software layer that sits between the user application and the ADI device.

The library is intended to serve multiple purposes:

1. Provide the application with a set of APIs that can be used to configure ADI device without the need for low-level register access. This makes it easy for the user to evaluate / use the device and removes the need for the user to know about every register and control sequence necessary to properly run the device.
2. Provide basic services to aid the application in controlling the components of the device module (if available), such as NCO configuration, JESD Link configuration or provide initialization sequence necessary to properly configure specific devices.
3. It also makes the application portable across different revisions of the hardware and even across different hardware modules.

The API does not, in any shape or form, alter the configuration or state of device on its OWN. It is the responsibility of the application to configure the part according to the required mode of operation, poll for status, etc. using the API. The library acts only as an abstraction layer between the application and the hardware.

As an example, the user application is responsible for the following:

- Implement platform dependent HAL functions like SPI, GPIO, Delay, LOGs, etc.
- Instantiating handles per target ADI device.
- Configuring the JESD Interface, DDC and NCOs.
- Calling Initialization and Configuration APIs in proper sequence.

The application should access the device only through the exported APIs. It is not recommended for the application to access the hardware device directly using direct SPI access. If the application chooses to directly access the ADI hardware this should be done in a very limited scope, such as for debug purposes and this practice may affect the reliability of the API functions.

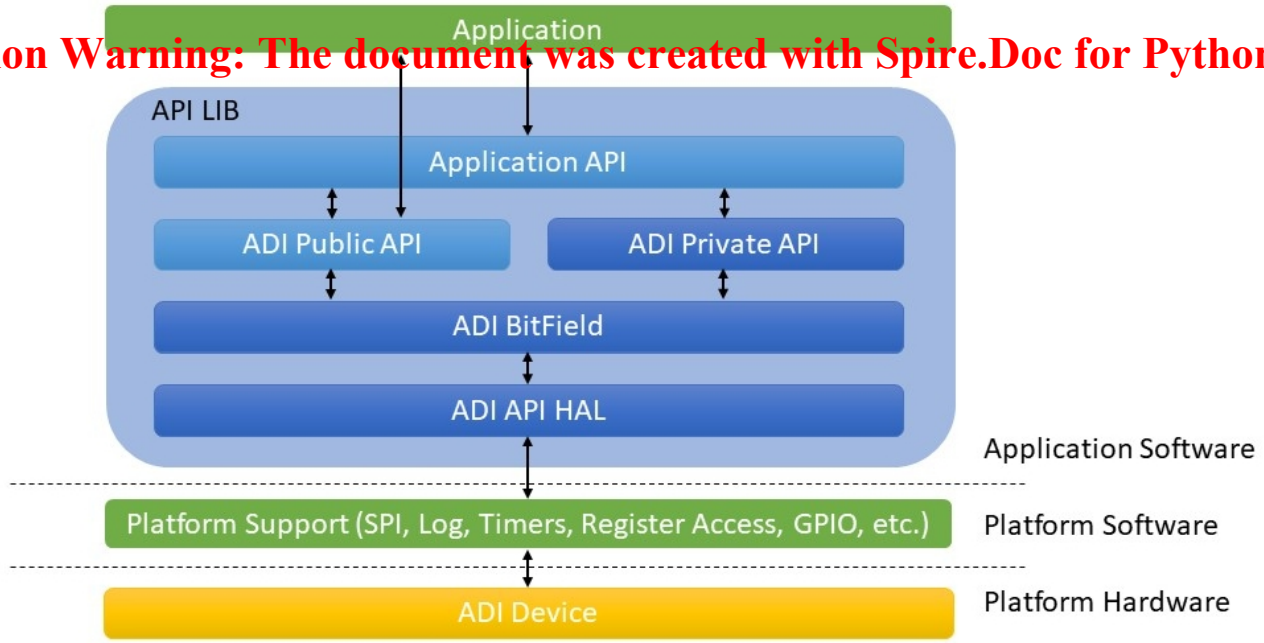

Figure 1. Shows a simple overview of the API Architecture.

Folder Structure

The collective files of the ADI's API library are structured as depicted in Figure 2. Each branch is explained in the following sections. The library is supplied in source format. All source files are in standard ANSI C to simplify porting to any platform.

Figure 2. ADxxxx Source Code Folder Structure.

Note: Here adxxxx refers to the ADI part number for which an API is available.

/adxxxx_api

The adxxxx_api root folder contains all the source code and example makefiles for an API.

/adxxxx_api/ ADXXXX

This folder includes the main API implementation code for a specific ADI device and any private header files used by the API. ADI maintains this code as intellectual property and all changes are at their sole discretion.

/adxxxx_api/ adi_inc

This folder contains all the public API interface files. These are the header files required by the client application for integration.

/adxxxx_api/ adi_utils

This folder contains ADI helper functions common to all ADI APIs, these functions are internal private functions not designed for use by client application.

/adxxxx_app

This folder contains simple source code examples of how to use the ADI's API for creating user application. The application targets the ADI's evaluation board platform. Customers can use this example code as a guide to develop their own application based on their requirements. This folder also contains **adxxxx_api** and all the subfolders within as explained above.

/Docs

Documentation for API is in this folder. This document, along with part specific API details, porting and integration guide, etc. can be found here.

API Interface

Overview

The header files listed in include folder, **/adxxxx_api/ adi_inc** describe public interface of a specific ADI device with the client application. It consists of several header files listed in Table 1.

File Name	Description	To be included in Client Application.
adi_adxxxx.h	Details ADxxxx API library exposed to client application. All the functionalities and control of that device is implemented as API functions and should be used by user application to access the device.	YES
adi_cms_api_common.h	Defines macros, enumerations and data structures used by all ADI API libraries. It also contains HAL pointer function declaration.	NO
adi_cms_api_config.h	Defines the various application configuration options for the API.	NO.

Table 1. ADxxxx API Interface.

Each API library for an ADI device will have a header file that lists its supported APIs that the client application may use to interface with the ADI device. For example, the adi_adxxxx.h header file lists all the APIs that are available to control and configure that specific device. The other header files are used for definitions and configurations that may be used by the API but not for client application. The subsequent sections will provide description for these header files.

adi_adxxxx.h

The ADI's API library has a main interface header file adi_adxxxx.h that defines the software interface to a specific ADI device. It consists of a list of API functions and several data structures and enumerations to describe the configurations and settings that are configurable on the target device.

Data Structures

The header file also defines the device handle ***adi_adxxxx_device_t*** which is a data structure that acts a software reference to an instance of an ADI device. This handle maintains a reference to HAL functions (***adi_adxxxx_hal_t***), configuration status of the chip (***adi_adxxxx_info_t***) and a GPIO structure (***adi_adxxxx_gpio_info_t***) that stores all the digital GPIO pins required to operate the chip. Table 2 summarizes these handles.

Structure Member	Description	User Read / Write Access	Required by API
hal_info	A structure maintaining a reference to the application's implementation of the HAL functions required by the API. See Table 3 for information in individual components of adi_adxxxx_hal_t.	Read / Write	Required
dev_info	A structure maintaining a reference to the configuration status of the device. See Table 4 for information in individual components of adi_adxxxx_info_t.	Read	Required
gpio_info	A structure that holds an array of digital GPIO pin information.	Read / Write	Required

Table 2. ADxxxx Device Handle Members.

hal_info

Here the HAL specific references shall be instantiated by the client application, initialized by the application with client specific data. The subsequent sections provide the information for correct integration.

All the platform specific members of the HAL structure must be configured by the client application prior to calling any API with the handle. A summary of the individual component members of HAL handle, **adi_adxxxx_hal_t,** supported by the API are listed in Table 3.

Structure Member	Description	User Read / Write Access	Required by API
user_data	Void Pointer to a user defined data structure. Shall be passed to all HAL functions.	Read / Write	Required
hw_open	Function pointer to platform initialization function for ADxxxx hardware.	Read / Write	Optional
hw_close	Function pointer to platform shutdown function for ADxxxx hardware.	Read / Write	Optional
spi_xfer	Function pointer to platform specific SPI data transfer function for the target device.	Read / Write	Required
gpio_write	Function pointer to platform specific function to set GPIO of the target device to 1 or 0.	Read / Write	Required
gpio_read	Function pointer to platform specific function to read GPIO state of the target device.	Read / Write	Required
log_write	Function pointer to log error, warnings and debug messages.	Read / Write	Optional
delay_us	Function pointer to delay function.	Read / Write	Required

Table 3. Members of adi_adxxxx_hal_t. ADI's device HAL handle.

dev_info

Similarly, a list of common individual component members of **adi_adxxxx_info_t** data structure is mentioned in Table 4.

Structure Member	Description	User Read / Write Access	Required by API
chip_type	Stores Chip Type number.	Read Only	Required
prod_id	Stores Product ID.	Read Only	Required
spi_rev	Stores SPI Register Map revision version.	Read Only	Required
variant	Stores chip variant version	Read Only	Required
feol	Stores Front End of Line number.	Read Only	Required
sif	Stores Stress Intensity Factor Version.	Read Only	Required
beol	Stores Back End of Line number.	Read Only	Required

Table 4. Members of adi_adxxxx_info_t. ADI's device info handle.

gpio_info

In the API, GPIOs are structured as nested data structures. The struct "adi_adxxxx_gpio_info_t" stores an array of digital GPIOs required for the working any ADI part. Members of this array are of struct "adi_adxxxx_gpio_t" type, which stores a GPIO id used to distinguish each GPIO and also marks the index location of that GPIO in the array. The void pointer points to a user defined struct that stores the information required to successfully control a GPIO. The reason for such implementation is because depending upon the user platform, a GPIO can be accessed and controlled in different ways. So, the user must provide corresponding information depending upon how its platform controls GPIO operation in its platform API. The void pointer can point to a Register location and the GPIO offset within that register or to an object pointer which can control the GPIO.

Refer to the adi_adxxxx_device_t section and the HAL Function Pointer Data Types section for full description and details on configuration.

Note:

Any API is not just limited to the above-mentioned data structures and its structure members. These are the most common structures that each API share. As per the part requirement new data structures and structure members along with multiple enumerators can be added by ADI. If so, its details would be documented in part specific API literature.

Struct Reference

adi_adxxxx_device_t

- ***adi_adxxxx_hal_t*** hal_info
  - void * user_data
  - adi_hw_open_t hw_open
  - adi_hw_close_t hw_close
  - adi_spi_xfer_t spi_xfer
  - adi_delay_us_t delay_us
  - adi_gpio_write_t gpio_write
  - adi_gpio_read_t gpio_read
  - adi_log_write_t log_write
  - .....
- ***adi_adxxxx_info_t*** dev_info
  - uint8_t chip_type
  - uint16_t prod_id
  - .....
- **adi_adxxxx_gpio_info_t** gpio_info
  - **adi_adxxxx_gpio_t** gpios[ADxxxx_NUM_GPIO_PINS]
    - adi_adxxxx_gpio_e gpio_id
    - void * info
- .....

adi_cms_api_common.h

This header contains all the definitions that are common to the ADI APIs.

All ADI APIs is designed to be platform agnostic. However, it requires access to platform functionality, such as SPI transactions, GPIO control, system delays, logging functions, etc. that the client application must implement and make available to the API. These functions are collectively referred to as the platform Hardware Abstraction Layer (HAL).

Function pointers to these HAL functions are defined by the API definition interface header file, **adi_cms_api_common.h**. The implementation of these functions is platform dependent and shall be implemented by the client application as per the client platform specific requirements. The client application shall assign each pointer to the required platform's HAL function implementation before calling any API Functions. Refer to the API Integration section for example.

The part specific device handle, adi_adxxxx_device_t, has a function pointer member for each of the HAL functions as shown in adi_adxxxx_hal _t handle. These function pointers are listed in Table 5 below. Refer to HAL Function Pointer for detail description of these function pointers.

Function Pointer Name	**Purpose** (Points to platform specific implementation of ….)	**Requirement**
* adi_spi_xfer_t	SPI Data transfer function.	Required
* adi_gpio_write_t	GPIO write function.	Required
* adi_gpio_read_t	GPIO read function.	Optional
* adi_delay_us_t	Wait / thread sleep function in micro-seconds.	Required
* adi_hw_open_t	Open and initialize resources and peripherals required by ADxxxx.	Optional
* adi_hw_close_t	Close and relinquish resources and peripherals required by ADxxxx.	Optional
* adi_log_write_t	Write error, warning, and error messages from API to target platform.	Optional

Table 5. HAL Function Pointers

HAL Function Pointers

All the HAL function pointer that connects any ADI API to target platform are mentioned below. The client application should make sure all the resources and peripheral initialization is achieved successfully before accessing any higher-level API functions.

Failure to implement these functions properly or connect the function pointers may result in unexpected API results.

The HAL function pointers required by API are listed below and explained in detail in further sections.

- typedef int32_t (*adi_hw_open_t) (void *user_data);
- typedef int32_t (*adi_hw_close_t) (void *user_data);
- typedef int32_t (*adi_spi_xfer_t) (void *user_data, uint8_t *in_data, uint8_t *out_data, uint32_t size_bytes);
- typedef int32_t (*adi_gpio_write_t) (void *user_data, uint32_t gpio, uint32_t value);
- typedef int32_t (*adi_gpio_read_t) (void *user_data, uint32_t gpio, uint32_t *value);
- typedef int32_t (*adi_delay_us_t) (void *user_data, uint32_t us);
- typedef int32_t (*adi_log_write_t) (void *user_data, int32_t log_type, const char *message, va_list argp);